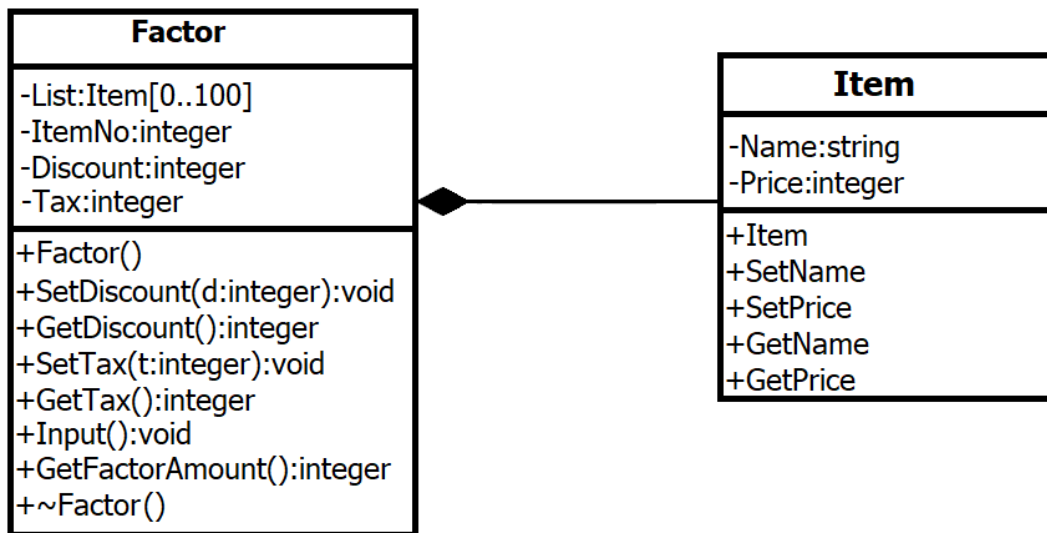


می خواهیم در یک سامانه حسابداری اطلاعات مربوط به فاکتورهای فروش یک فروشگاه را نگهداری کنیم. برای این منظور دو کلاس Item و Factor طراحی شده است. نمودار کلاس زیر بخشی از نمودار کلاس طراحی شده برای این سامانه است.



کلاس Item اقلام موجود در فروشگاه را نشان می دهد که دو ویژگی نام (Name) و قیمت (Price) دارد. این کلاس با توجه به اعضای داده ای و توابع نشان داده شده در نمودار کلاس پیاده سازی شده است. سازنده پیش فرض مقدار "" را به متغیر Name و 0 را به متغیر Price نسبت می دهد (نیازی به پیاده سازی این کلاس وجود ندارد).

کلاس Factor فاکتور فروش را نشان می دهد که دارای ۴ ویژگی است. List آرایه ای از نوع کلاس Item که فهرستی از اقلام خریداری شده را نشان می دهد. ItemNo تعداد اقلام فاکتور است که مقدار اولیه آن صفر است و حداکثر ۱۰۰ قلم کالا به فاکتور می توان اضافه کرد. Discount درصد تخفیف فاکتور است و مقدار پیش فرض برای آن صفر می باشد و در نهایت Tax مالیات بر ارزش افزوده را نمایش می دهد که مقدار آن ۹ درصد برای کلیه فاکتورها می باشد (متغیر ایستا).

۱- تعریف کلاس Factor را بدون پیاده سازی توابع بنویسید. دقت کنید با توجه به عملکرد توابع، توابع ثابت و ایستا را هم به شکل مناسب درون کلاس مشخص کنید. (دقت کنید تغییرات لازم با توجه به سوالات بعدی نیز در کلاس Factor اعمال شود). (۲ نمره)

۲- تابع سازنده بدون پارامتری پیاده سازی کنید که مقادیر اولیه را به متغیرهای کلاس نسبت دهد. متغیر ایستای Tax را نیز به شکل مناسب مقداردهی نمایید. (۱ نمره)

۳- تابع SetDiscount و GetDiscount را پیاده سازی کنید. دقت کنید که درصد تخفیف عددی بین ۰ و ۱۰۰ است در صورت نامعتبر بودن مقدار، درصد تخفیف را صفر قرار دهید. (۱ نمره)

۴- تابع Input برای دریافت تعداد اقلام فاکتور، لیست اقلام فاکتور و همین طور درصد تخفیف از ورودی طراحی شده است. این تابع را پیاده سازی کنید و مقادیر را از ورودی دریافت کنید و داده های کلاس Factor را به شکل مناسب مقداردهی کنید. (۱/۵ نمره)

۵- تابع GetFactorAmount برای محاسبه مبلغ فاکتور با توجه به لیست اقلام فاکتور و قیمت آنها، درصد تخفیف و درصد مالیات بر ارزش افزوده طراحی شده است. این تابع را برای کلاس Factor پیاده سازی کنید. (۱ نمره)

۶- عملگر << (درج در جریان) را برای کلاس بازتعریف کنید تا لیست اقلام فاکتور را به همراه مبلغ فاکتور چاپ کند (۱ نمره)

۷- عملگر + را برای کلاس بازتعریف کنید تا با جمع دو Factor فاکتور جدیدی شامل اقلام هر دو فاکتور ایجاد و برگرداند. تعداد اقلام مجموع تعداد اقلام دو فاکتور و درصد تخفیف برابر درصد تخفیف بیشتر باشد. (۱/۵ نمره)

۸- برنامه ای بنویسید تا یک نمونه از شی Factor ایجاد کند؛ سپس با استفاده از توابع پیاده سازی شده اطلاعات مربوط به فاکتور را از ورودی دریافت کرده و سپس لیست اقلام فاکتور و مبلغ فاکتور را محاسبه و چاپ کند. (۱ نمره)

می‌خواهیم کلاسی برای نگهداری و انجام عملیات مختلف، روی مجموعه‌ای از اعداد صحیح ایجاد کنیم. این مجموعه می‌تواند اعداد صحیح مثبت کمتر ۱۰۰ را در خود جای دهد. برای نگهداری این مجموعه از آرایه‌ای از 0 و 1 (Elements) استفاده می‌کنیم. [i]Elements برابر 1 خواهد بود اگر i در مجموعه وجود داشته باشد و 0 خواهد بود اگر i در مجموعه وجود نداشته باشد.

```
class IntegerSet
{private:
    int Elements[100];
    ...
};
```

- ۱- برای کلاس IntegerSet سازنده (constructor) پیاده‌سازی کنید که اگر پارامتر ورودی ارسال نشود یک مجموعه تهی ایجاد کند و اگر یک عدد صحیح به عنوان ورودی ارسال شود، مجموعه‌ای دارای تمامی اعداد صحیح مثبت کمتر از ورودی ایجاد کند. (۱ نمره)
- ۲- برای کلاس IntegerSet مخربی (destructor) پیاده‌سازی کنید که عناصر مجموعه را چاپ کند. (۱ نمره)
- ۳- متد InsertElement را برای کلاس IntegerSet پیاده‌سازی کنید که عنصری را به عنوان پارامتر ورودی دریافت کند و آن را به مجموعه اضافه کند. (۱ نمره)

```
void InsertElement (int x)
```

- ۴- متد IsElement را برای کلاس IntegerSet پیاده‌سازی کنید که عنصری را به عنوان پارامتر ورودی دریافت کند و بررسی کند که این عنصر به مجموعه تعلق دارد یا نه. (در صورت تعلق داشتن true و در صورت تعلق نداشتن false را برگرداند). (۱ نمره)

```
bool IsElement (int x)
```

- ۵- اپراتور << را برای کلاس IntegerSet تعریف مجدد (overload) کنید تا عناصر مجموعه را به شکل مرسوم مجموعه‌ها در ریاضیات چاپ کند. (مثال: {1,2,10}). (۱ نمره)
- ۶- اپراتور == را برای کلاس IntegerSet تعریف مجدد (overload) کنید تا برابر بودن دو مجموعه را بررسی کند. (۱ نمره)
- ۷- اپراتور + را برای کلاس تعریف مجدد (overload) کنید که اجتماع دو مجموعه را محاسبه کند و در قالب IntegerSet برگرداند. (۱ نمره)
- ۸- اپراتور * را برای کلاس تعریف مجدد (overload) کنید که اشتراک دو مجموعه را محاسبه کند و در قالب IntegerSet برگرداند. (۱ نمره)
- ۹- با توجه به متدها و عملگرهای اضافه شده تعریف کلاس را بازنویسی کنید. (۱ نمره)
- ۱۰- خروجی قطعه کد زیر را مشخص کنید. (۱ نمره)

```
int main()
{ IntegerSet S1,S2(10);
    S1.InsertElement(7); S1.InsertElement(50); S1.InsertElement(200);
    S2.InsertElement(20);
    cout << S1 << endl << S2 <<endl << S1+S2 << endl << S1*S2 << endl;
    if (S1 == S2) cout << "Equal";
    else cout << "Not Equal";
    if ((S1*S2).IsElement(7)) cout <<"YES";
    else cout << "NO"
}
```

لطفا برای پاسخگویی به سوالات نکات زیر را با دقت خوانده و این نکات را در زمان پاسخگویی رعایت کنید:

- نکته ۱: در مورد کلاس خواسته شده، تعریف کلاس (class declaration) را به صورت جداگانه بنویسید و توابع عضو را خارج از کلاس پیاده سازی کنید. دقت کنید که کل موارد خواسته شده در کلاس را مطالعه کنید و همه موارد خواسته شده را در تعریف کلاس قرار دهید.
- نکته ۲: توابعی عضوی که می توانید به صورت ثابت (const) تعریف کنید.
- نکته ۳: فرض کنید کلاس Date و با توابع زیر پیاده سازی شده است (مثال کلاس) و نیازی به پیاده سازی توابع ذکر شده نیست. در صورت نیاز به توابع دیگر می توانید توابع مورد نیاز را به کلاس Date اضافه کرده و پیاده سازی کنید.

```
class Date
{
    friend ostream &operator << (ostream &, const Date &);
    friend istream &operator >> (istream &, Date &);
public:
    Date(int =20,int =2,int =1404);
    void setDate(int, int, int);
    int getDay ( ) const;
    int getMonth ( )const;
    int getYear ( ) const;
    void print ( ) const;
    Date operator ++ ( );
    Date operator ++ (int);
    Date operator += (int);
    bool operator == (const Date &) const;
    bool operator < (const Date &) const;
    bool operator > (const Date &) const;
private:
    int day;
    int month;
    int year;
    static int DaysOfMonth[];
};
```

یک فروشگاه مواد غذایی می خواهد سامانه ای برای نگهداری اطلاعات مواد غذایی فروشگاه خود ایجاد کند.

۱- کلاس FoodItem را با توجه به موارد خواسته شده پیاده سازی کنید:

الف) کلاس دارای ویژگیهای خصوصی زیر باشد:

نام (Name): رشته، قیمت (Price): عدد اعشاری، تاریخ تولید (PDate): شئی از کلاس Date، ماندگاری (SLife): عدد صحیح (تعداد ماهی که ماده غذایی بعد از تولید قابل مصرف است)

ب) تابع سازنده ای برای مقداردهی اولیه به داده ها داشته باشد (مقادیر داده ها به عنوان پارامتر ورودی تابع در نظر گرفته شود) و تابع سازنده پیش فرض مقادیر "food", 0, 1404/2/20 و 1 را به ترتیب به داده های کلاس نسبت دهد.

پ) تابع SetPrice و GetPrice داشته باشد که برای مقداردهی و دستیابی به متغیر Price استفاده شود. بدیهی است که مقدار Price باید مثبت باشد. (با توجه به محدودیت زمان امتحان نیازی به پیاده سازی سایر توابع دستیابی و مقدار دهی نیست)

ت) تابع به نام getExpDate برای محاسبه تاریخ انقضای محصول داشته باشد که بر اساس تاریخ تولید و ماندگاری ماده غذایی تاریخ انقضا محاسبه و برگرداند.

ث) عملگر >> را برای دریافت ویژگیهای Name، Price، PDate و SLife از ورودی بازتعریف کنید.

ج) عملگر << را برای نمایش اطلاعات کلاس FoodItem به فرم زیر بازتعریف کنید (به جای نقطه چین ها مقادیر ویژگیها چاپ شود).

Name = ... Price = ... Production Date = ../../.. Expired Date = ../../..

چ) عملگر < را بازتعریف کنید که یک شیء FoodItem را به عنوان عملوند چپ و یک شیء Date به عنوان عملوند راست بپذیرد و اگر تاریخ انقضای FoodItem قبل از تاریخ Date باشد، true و در غیر این صورت false را برگرداند.

۲- در تابع main آرایه ای ۱۰۰ عنصری از کلاس FoodItem ایجاد کرده، اطلاعات مربوط به تمامی اشیا آرایه را از ورودی دریافت کنید و سپس با مقایسه تاریخ انقضای عناصر آرایه با تاریخ امروز، اطلاعات مربوط به تمامی FoodItem هایی که تاریخ انقضای آنها گذشته است را چاپ کنید.

یک آزمایشگاه تحقیقاتی نیاز به محاسبات مختلف روی اعداد صحیح بسیار بزرگ مثبت (حداکثر ۵۰ رقمی) دارد. چون انواع داده اصلی زبان C++ نمی‌تواند این اعداد را پردازش کند، می‌خواهیم کلاسی برای نگهداری و انجام عملیات روی این اعداد ایجاد کنیم. برای ذخیره این اعداد از آرایه‌ای استفاده کرده‌ایم تا ارقام درون آن نگهداری شود.

```
class HugeInt
{private:
    int number[50];
    static const short int MaxDigits = 50;
    ...
};
```

توجه: تمامی توابع کلاس را بیرون از بدنه کلاس پیاده‌سازی کرده و توابعی که امکان ثابت تعریف کردن آنها وجود دارد، به صورت ثابت پیاده‌سازی کنید. اعلان کلاس در سوال ۸ از شما خواسته شده است پس در سوالات ۱ تا ۷ تنها پیاده‌سازی توابع کافی است.

۱- برای کلاس HugeInt سازنده‌ای (constructor) پیاده‌سازی کنید اگر پارامتر ورودی ارسال نشود، تمامی ارقام را برابر صفر قرار دهد و اگر یک رشته‌ای شامل عدد به عنوان ورودی ارسال شود، ارقام آن را مشابه مثال زیر در آرایه قرار دهد. (۱/۵ نمره)

	0		38	39	40	41	42	43	44	45	46	47	48	49
"95783002614"	0	...	0	9	5	7	8	3	0	0	2	6	1	4

- ۲- تابع Digit را برای کلاس HugeInt پیاده‌سازی کنید که تعداد ارقام را برگرداند. (برای مثال فوق ۱۱ برگرداند) (۱ نمره)
- ۳- اپراتور << را برای کلاس HugeInt تعریف مجدد (overload) کنید تا عدد را به فرم صحیح چاپ کند. (۱ نمره)
- ۴- اپراتور ++ پسوندی را برای کلاس HugeInt تعریف مجدد (overload) کنید تا یک واحد به عدد اضافه کند. (۱ نمره)
- ۵- اپراتور + را برای کلاس تعریف مجدد (overload) کنید که مجموع دو عدد را محاسبه و در قالب HugeInt برگرداند. (۱/۵ نمره)
- ۶- برای کلاس HugeInt مخربی (destructor) پیاده‌سازی کنید که پیغام "number is destructed!" را چاپ کند. (۱ نمره)
- ۷- تابع ایستایی (static) را برای کلاس HugeInt پیاده‌سازی کنید که حداکثر تعداد ارقام ممکن برای کلاس را برگرداند. (۱ نمره)
- ۸- با توجه به متدها و عملگرهای اضافه شده اعلان کلاس را تکمیل کنید. (۱ نمره)
- ۹- خروجی قطعه کد زیر را مشخص کنید. (۱ نمره)

```
int main()
{
    HugeInt num1("456360000"), num2("999999999"), num3;
    num3 = num1 + num2;
    cout << num1 << '-' << num2 << '+' << num3;
}
```

موفق باشید

یک موسسه آموزشی می خواهد اطلاعات مربوط به دوره های آموزشی فشرده خود را با استفاده از یک سامانه کامپیوتری مدیریت کند. برای این منظور کلاس Course طراحی شده است. این کلاس دارای ۴ ویژگی است. ویژگی Title یک string است که عنوان دوره را نگهداری می کند. ویژگی Attendees آرایه ای از نوع string است و اسامی افراد شرکت کننده را نگهداری می کند. حداکثر ظرفیت هر دوره ۲۰ نفر است. ویژگی Duration تعداد جلسات دوره را نمایش می دهد که فعلا برای تمامی دوره ها ثابت و برابر ۱۰ جلسه است (ویژگی ایستا). ویژگی StartDate تاریخ شروع دوره را ذخیره می کند. جلسات دوره با توجه به فشرده بودن دوره به صورت روزانه و حتی در روزهای تعطیل هم برگزار می شود. فرض کنید کلاس Date و توابع زیر پیاده سازی شده است و نیازی به پیاده سازی توابع ذکر شده نیست.

```
class Date
{
friend ostream &operator <<(ostream &,const Date &);
public:
    Date(int =1,int =1,int =1403);
    void setDate(int, int, int);
    int getDay() const;
    int getMonth()const;
    int getYear() const;
    void print() const;
    Date operator ++();
    Date operator ++(int);
    Date operator +=(int);
private:
    int day;
    int month;
    int year;
    static int DaysofMonth[];
};
```

- ۱- تابع سازنده ای برای کلاس Course پیاده سازی کنید که با دریافت یک رشته برای عنوان دوره و یک شی Date داده های کلاس را مقدار دهی اولیه کند. کد لازم برای مقدار دهی اولیه داده ایستای کلاس (Duration) را هم بنویسید. (۱ نمره)
- ۲- توابع ایستای SetDuration و GetDuration را برای کلاس Course پیاده سازی کنید. دقت کنید که تعداد جلسات مقداری مثبت است که در صورت نادرست بودن مقدار جدید جایگزین مقدار قبلی نخواهد شد. (۱ نمره)
- ۳- تابع SetStartDate و GetStartDate را برای کلاس Course پیاده سازی کنید. تابع SetStartDate با دریافت سه عدد روز، ماه و سال به عنوان پارامتر ورودی تاریخ جدید را به تاریخ شروع دوره نسبت دهد. (۱/۵ نمره)
- ۴- تابع GetEndDate برای کلاس Course پیاده سازی کنید که تاریخ اتمام دوره را برگرداند (تاریخ اتمام دوره با توجه به تاریخ شروع و تعداد جلسات دوره محاسبه می گردد). (۱/۵ نمره)
- ۵- عملگر >> را برای کلاس Course بازتعریف کنید تا با دریافت تعداد افراد شرکت کننده، نام افراد شرکت کننده را از ورودی دریافت کرده و آنها را در آرایه Attendees ذخیره کند. (۱/۵ نمره)
- ۶- عملگر << (درج در جریان) را برای کلاس Course بازتعریف کنید تا عنوان دوره، لیست شرکت کننده ها را به همراه تاریخ شروع و اتمام دوره را چاپ کند. (۱/۵ نمره)
- ۷- تعریف کلاس Course را بدون پیاده سازی توابع درون آن بنویسید. لیست داده ها و توابع کلاس در توضیحات اولیه و سوالات آورده شده است. دقت کنید توابع ثابت و ایستا و دوست را به شکل مناسب درون کلاس مشخص کنید. (۲ نمره)
- ۸- برنامه ای بنویسید که یک شی از کلاس Course با عنوان دوره "Photo Shop" و تاریخ شروع ۱۴۰۳/۱۰/۶ ایجاد کند. سپس لیست شرکت کننده ها را از ورودی دریافت کند. سپس با تغییر تاریخ شروع دوره به ۱۴۰۳/۳/۶ مشخصات دوره (عنوان دوره، لیست شرکت کننده ها، تاریخ شروع و اتمام دوره) را چاپ کند. (۲ نمره)

یک استاد دانشگاه می‌خواهد اطلاعات مربوط به دروس خود را نگهداری و پردازش کند، می‌خواهیم کلاسی برای نگهداری و انجام عملیات روی این اطلاعات ایجاد کنیم. این کلاس دارای داده‌های زیر است. (جلوی هر داده توضیحات لازم داده شده است).

```
class Course
```

```
{private:
```

```
    static const int MaxSize = 60; // سقف تعداد دانشجویان مجاز برای هر درس
```

```
    string name; // نام درس
```

```
    int size; // تعداد دانشجویان کلاس
```

```
    string studentNames[MaxSize]; // نام دانشجویان درس
```

```
    float examGrades[MaxSize]; // نمرات امتحان (عددی بین ۰ تا ۲۰)
```

```
    float homeworkGrades[MaxSize]; // نمرات تکالیف (عددی بین ۰ تا ۲۰)
```

```
    float homeworkWheight; // میزان تاثیر نمره تکالیف در نمره نهایی (عددی بین ۰ تا ۱)
```

```
    ... // بدیهی است که میزان تاثیر نمره امتحان با کسر میزان تاثیر تکالیف از یک قابل محاسبه است.
```

```
};
```

توجه: تمامی توابع کلاس را بیرون از بدنه کلاس پیاده‌سازی کرده و توابعی که امکان ثابت تعریف کردن آنها وجود دارد، به صورت ثابت پیاده‌سازی کنید. تعریف کلاس در سوال ۸ از شما خواسته شده است پس در سوالات ۱-۷ تنها پیاده‌سازی توابع کافی است.

- ۱- برای کلاس Course سازنده‌ای (constructor) پیاده‌سازی کنید که نام درس، میزان تاثیر نمره تکالیف به عنوان پارامتر دریافت کند و در صورت مجاز بودن مقادیر آنها را به متغیرهای مربوطه نسبت دهد. (۱ نمره)
- ۲- تابع set و get را برای متغیر HomeworkWeight پیاده‌سازی کنید. (۱ نمره)
- ۳- تابع Insert را برای کلاس Course پیاده‌سازی کنید تا نام دانشجو، نمره تکالیف، نمره امتحان را به عنوان پارامتر دریافت کرده و این اطلاعات را به کلاس اضافه نماید. (۱ نمره)
- ۴- تابع getFinalGrade را برای کلاس Course پیاده‌سازی کنید تا با دریافت نام دانشجو و پیدا کردن اطلاعات دانشجو (نمره تکالیف و نمره امتحان) نمره نهایی دانشجو از ۲۰ نمره را محاسبه و برگرداند. (۱/۵ نمره)
- ۵- اپراتور ++ پسوندی را برای کلاس Course تعریف مجدد (overload) کنید به شکلی که یک نمره به نمرات امتحان تمامی دانشجویان اضافه کند. (۱ نمره)
- ۶- اپراتور << را برای کلاس تعریف مجدد (overload) کنید که اسامی تمامی دانشجویان را به همراه نمره نهایی آنها چاپ کند. (۱/۵ نمره)
- ۷- تابع ایستایی (static) را برای کلاس Course پیاده‌سازی کنید که حداکثر تعداد دانشجویان هر درس را برگرداند. (۱ نمره)
- ۸- با توجه به متدها و عملگرهای اضافه شده تعریف کلاس را بازنویسی کنید. (۱ نمره)

۹- برنامه‌ای بنویسید تا نمونه‌ای از کلاس Course برای درس C++ Programming ایجاد کند. در این درس میزان تاثیر تکالیف، 0.4 (40 درصد) و میزان تاثیر نمرات امتحان 0.6 (60 درصد) است. این درس ۴۰ دانشجو دارد. با استفاده از توابع پیاده‌سازی شده اطلاعات دانشجویان را از ورودی دریافت کنید و سپس نمرات نهایی کلاس را چاپ کنید. (۱ نمره)

می‌خواهیم کلاسی برای نگهداری و انجام عملیات مختلف روی مجموعه‌ای از اعداد حقیقی مرتب ایجاد کنیم. آرایه List اعداد حقیقی و n تعداد عناصر را نگهداری می‌کند. اعداد باید به صورت مرتب در List نگهداری شود.

```
class SortedList
{private:
    double List[1000];
    int n;
    ...
};
```

۱- برای کلاس SortedList سازنده (constructor) و مخربی (destructor) پیاده‌سازی کنید که در ابتدای ایجاد شی، یک لیست خالی ایجاد و در زمان از بین رفتن شی، عناصر لیست را به صورت مرتب چاپ کند. (۱ نمره)

۲- متد insert را برای کلاس SortedList پیاده‌سازی کنید که عنصری را به عنوان پارامتر ورودی دریافت کند و آن را به مجموعه اعداد اضافه کند. (دقت کنید اعداد در لیست به صورت مرتب نگهداری شود.) (۲ نمره)

```
void insert (double x)
```

۳- متد search را برای کلاس SortedList پیاده‌سازی کنید که عنصری را به عنوان پارامتر ورودی دریافت کند و جایگاه آن عنصر در لیست را به عنوان خروجی برگرداند. (در صورت عدم وجود داده مقدار ۱- را برگرداند) (۱ نمره)

```
int search (double x)
```

۴- اپراتور << را برای کلاس SortedList تعریف مجدد (overload) کنید تا عناصر لیست را به صورت مرتب با قرار دادن فاصله بین اعداد چاپ کند. (۱ نمره)

۵- اپراتور == را برای کلاس SortedList تعریف مجدد (overload) کنید تا برابر بودن دو شی از کلاس SortedList را بررسی کند. (۱ نمره)

۶- اپراتور + را برای کلاس تعریف مجدد (overload) کنید که دو لیست مرتب را به هم الحاق نموده و لیست مرتبی ایجاد کند. (۲ نمره)

۷- با توجه به متدها و عملگرهای اضافه شده تعریف کلاس را بازنویسی کنید. (۱ نمره)

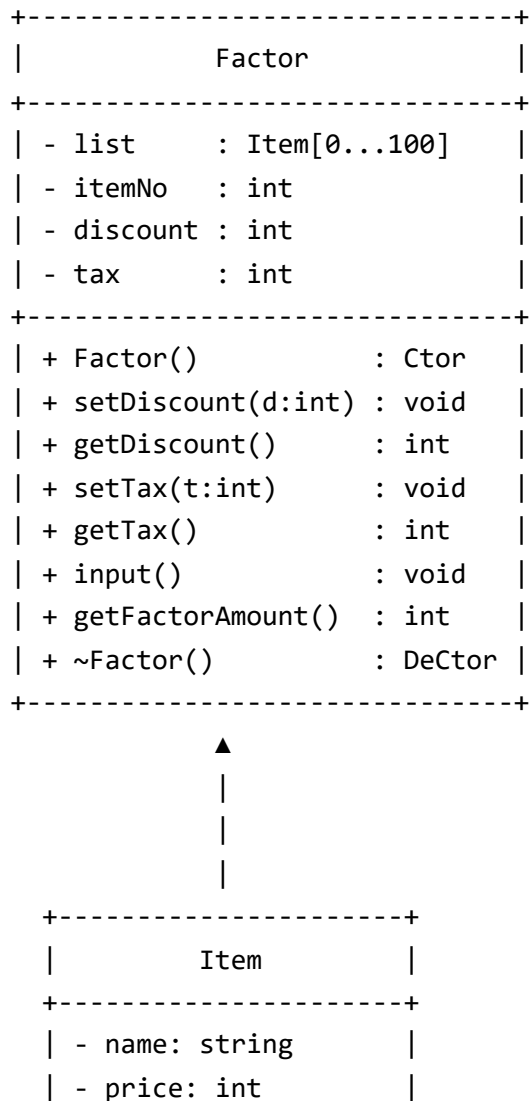
۸- خروجی قطعه کد زیر را مشخص کنید. (۱ نمره)

```
int main()
{ SortedList L1,L2;
    L1.insert(2.3);
    L1.insert(1.2);
    L1.insert(3.5);
    L2.insert(1.7);
    L2.insert(5.6);
    cout << L1 << ' ' << L2 << ' ' << L1.search(1.7) << ' ' << L2.search(1.7);
    if (L1 == L2) cout << "Equal";
        else cout << "Not Equal";
    L2 = L1 + L2;
}
```

Factor

- محدودیت زمان: ۱ ثانیه
- محدودیت حافظه: ۲۵۶ مگابایت

می خواهیم در یک سامانه حسابداری اطلاعات مربوط به فاکتور های فروش یک فروشگاه را نگهداری کنیم. برای این منظور دو کلاس *Item* و *Factor* طراحی شده است. نمودار کلاسی زیر بخشی از نمودار کلاسی طراحی شده در این سامانه است.




```

+-----+
| + Item()    : Ctor    |
| + setName() : void    |
| + setPrice(): void    |
| + getName() : string  |
| + getPrice(): int     |
+-----+

```

▼ توضیح دیاگرام UML

دیاگرام Unified Modeling Language یک دیاگرام مستطیل شکل برای نشان دادن ویژگی ها و توابع کلاس است که از سه بخش Class Name, Private Members, Public Members تشکیل شده.

۱. بخش بالای دیاگرام نام کلاس قرار می گیرد.

۲. در بخش وسط، اعضای خصوصی (private) کلاس لیست می شوند به طوری که قبل از نام هر کدام یک - (minus sign) قرار میگیرد که نشان دهنده private بودن آن عضو (تابع یا متغیر) از کلاس است و همچنین جلوی هر عضو یک : (colon) قرار گرفته و نوع داده ای (data type) آن عضو را مشخص می کند.

۳. بخش پایین دیاگرام نیز اعضای public کلاس را با قرار دادن یک + (plus sign) قبل از هر عضو نشان می دهد.

\

روابط بین هر کلاس در دیاگرام UML را با فلش (پیکان) هایی که از یک دیاگرام به دیاگرام دیگر خارج می شود نشان می دهند.

به عنوان مثال رابطه ترکیب دو کلاس، با یک فلش (پیکان) به سمت کلاسی که یک شی از کلاس دیگر در آن نمونه سازی شده است نشان داده می شود (مانند سوال).

کلاس *Item* اقلام موجود در فروشگاه را نشان می دهد که دو ویژگی نام (*name*) و قیمت (*price*) دارد. این کلاس با توجه به اعضای داده ای و توابع نشان داده شده در نمودار کلاس پیاده سازی شده است. سازنده پیش فرض مقدار "" را به متغیر *name* و 0 را به متغیر *price* نسبت می دهد. (نیازی به پیاده سازی این کلاس وجود ندارد).

کلاس *Factor* فاکتور فروش را نشان می دهد که دارای 4 ویژگی است.

۱. صفت *list* آرایه ای از نوع کلاس *Item* است که فهرستی از اقلام خریداری شده را نشان می دهد.
۲. صفت *itemNo* تعداد اقلام فاکتور است که مقدار اولیه آن صفر است و حداکثر 100 کالا را می توان به فاکتور اضافه کرد.
۳. صفت *discount* درصد تخفیف فاکتور است و مقدار پیش فرض برای آن صفر می باشد.
۴. و در نهایت صفت *tax* مالیات بر ارزش افزوده را نشان می دهد که مقدار آن 9 درصد برای کلیه فاکتور ها می باشد (متغیر ایستا).

توابع زیر را پیاده سازی کنید:

۱. تعریف کلاس *Factor* را بدون پیاده سازی توابع بنویسید. دقت کنید با توجه به عملکرد توابع، توابع ثابت و ایستا را هم به شکل مناسب درون کلاس مشخص کنید. (دقت کنید تغییرات لازم با توجه به سوالات بعدی نیز در کلاس *Factor* اعمال شود.)
۲. تابع سازنده بدون پارامتری پیاده سازی کنید که مقادیر اولیه را به متغیر های کلاس نسبت دهد. متغیر ایستای *tax* را نیز به شکل مناسب مقدار دهی نمایید.
۳. تابع *setDiscount* و *getDiscount* را پیاده سازی کنید. دقت کنید که درصد تخفیف عددی بین 0 و 100 است؛ در صورت نامعتبر بودن درصد تخفیف را صفر قرار دهید.
۴. تابع *input* برای دریافت تعداد اقلام فاکتور، لیست اقلام فاکتور و همین طور درصد تخفیف از ورودی طراحی شده است. این تابع را پیاده سازی کنید و مقادیر را از ورودی دریافت کنید و داده های کلاس *Factor* را به شکل مناسب مقدار دهی کنید.
۵. تابع *getFactorAmount* برای محاسبه مبلغ فاکتور با توجه به لیست اقلام فاکتور و قیمت آن ها، درصد تخفیف و درصد مالیات بر ارزش افزوده طراحی شده است. این تابع را برای کلاس *Factor* پیاده سازی کنید.
۶. عملگر $>>$ (درج در جریان) یا همان *ostream* را برای کلاس بازتعریف کنید تا لیست اقلام فاکتور را به همراه مبلغ فاکتور چاپ کند.
۷. عملگر $+$ را برای کلاس بازتعریف کنید تا با جمع دو فاکتور جدید، شامل اقلام هر دو فاکتور ایجاد و برگرداند. تعداد اقلام مجموع تعداد اقلام دو فاکتور و درصد تخفیف برابر درصد تخفیف بیشتر باشد.

۸. برنامه ای بنویسید تا یک نمونه از شی فاکتور ایجاد کند؛ سپس با استفاده از توابع پیاده سازی شده اطلاعات مربوط به فاکتور را از ورودی دریافت کرده و سپس لیست اقلام فاکتور را محاسبه و چاپ کند.